



خلاصه

در این آزمایش دو مدار ترکیبی در Logisim Evolution پیاده‌سازی شده‌اند که یک عدد BCD چهاررقمی را می‌گیرند و مشخص می‌کنند آیا مضرب ۳ (mod3_checker) یا مضرب ۱۱ (mod11_checker) است. فقط از گیت‌های پایه استفاده شده و مدار به صورت سلسله‌مراتبی از بلوک‌های full_adder، adder5/adder4 و مقایسه‌گرها ساخته شده است.

مضرب ۳ - جمع ارقام

چون $10 \equiv 1 \pmod{3}$ ، داریم

$$N = 1000D_3 + 100D_2 + 10D_1 + D_0 \equiv D_3 + D_2 + D_1 + D_0 \pmod{3}.$$

پس کافی است $S = D_3 + D_2 + D_1 + D_0$ را محاسبه کنیم و ببینیم $S \equiv 0 \pmod{3}$ هست یا نه. چون $2 \equiv -1 \pmod{3}$ ، بیت‌های زوج و فرد S وزن +۱ و -۱ دارند. با تعریف

$$P = s_0 + s_2 + s_4, \quad Q = s_1 + s_3 + s_5,$$

داریم $S \equiv P - Q \pmod{3}$. شرط $S \equiv 0 \pmod{3}$ معادل $P \equiv Q \pmod{3}$ است؛ یعنی $P = Q$ یا $\{P, Q\} = \{0, 3\}$. خود P و Q با یک full_adder روی سه بیت متناظر به دست می‌آیند.

مضرب ۱۱ - جمع متناوب ارقام

چون $10 \equiv -1 \pmod{11}$:

$$N \equiv (D_0 + D_2) - (D_1 + D_3) \pmod{11}.$$

با $A = D_0 + D_2$ و $B = D_1 + D_3$ ، بازه‌ی $A - B$ بین -۱۸ تا +۱۸ است و تنها مضرب‌های ۱۱ در این بازه -۱۱، ۰ و +۱۱ هستند. بنابراین

$$N \equiv 0 \pmod{11} \iff A + 11 = B \text{ یا } A = B \text{ یا } A = B + 11.$$

این سه شرط با جمع‌کننده (برای $A + 11$ و $B + 11$) و مقایسه‌گر تساوی پیاده می‌شوند.

تمام‌جمع‌کننده‌ی ۱ بیتی (full_adder)

پایه	جهت	پهنا
C_{in}, B, A	ورودی	۱ بیت
C_{out}, S	خروجی	۱ بیت

مدار با ۲ گیت XOR، ۲ گیت AND و ۱ گیت OR ساخته شده - بلوک پایه‌ی تمام محاسبات این آزمایش:

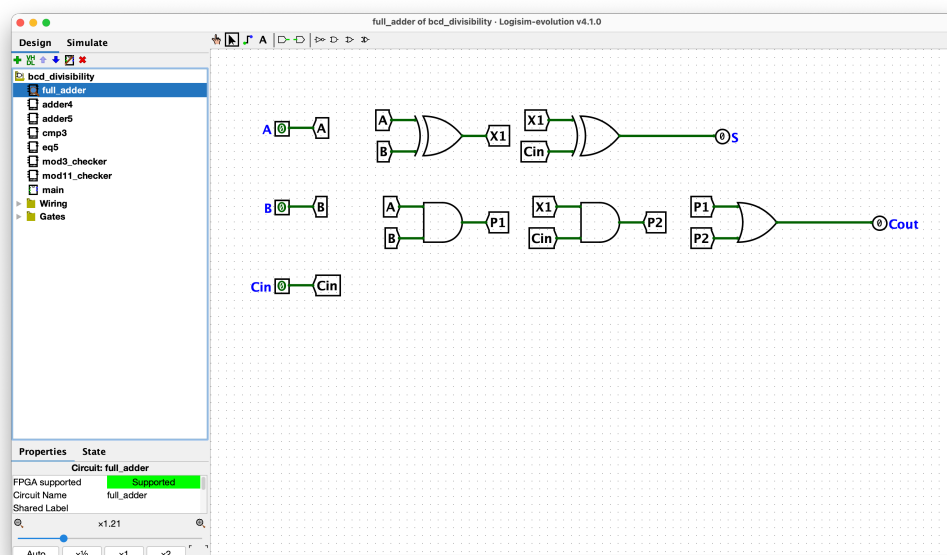
• $X_1 = A \oplus B$ - نیمه‌جمع، بدون در نظر گرفتن رقم نقلی ورودی.

• $P_1 = A \wedge B$ - آیا از جمع A و B به‌تنهایی رقم نقلی تولید می‌شود؟

• $S = X_1 \oplus C_{in}$ - جمع نهایی سه ورودی.

• $P_2 = X_1 \wedge C_{in}$ - آیا از جمع X_1 و C_{in} رقم نقلی تولید می‌شود؟

• $C_{out} = P_1 \vee P_2$ - رقم نقلی خروجی نهایی.



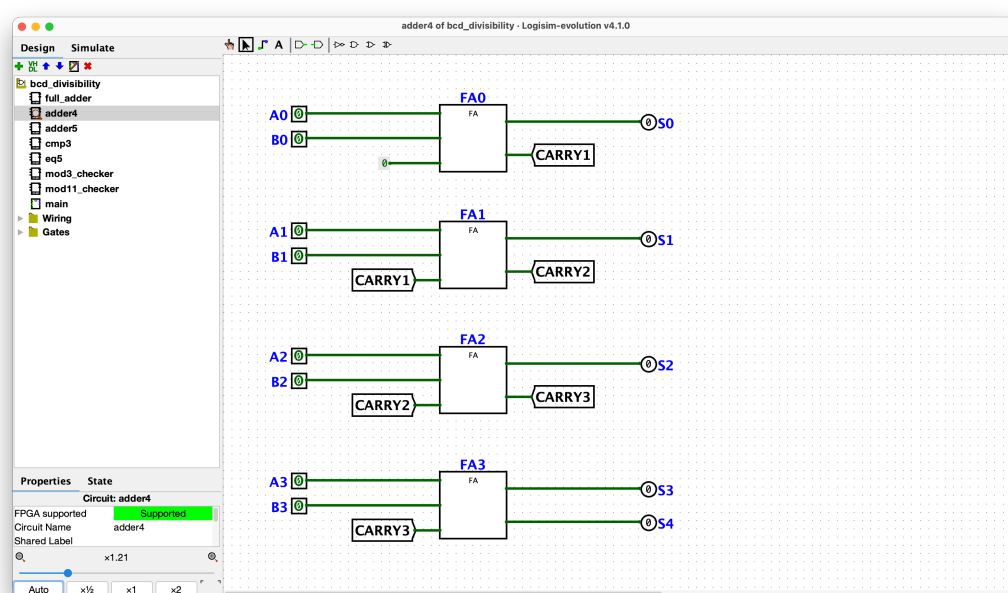
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

جمع‌کننده ۴ بیتی (adder4)

پایه	جهت	پهنا
B0..B3, A0..A3	ورودی	۱ بیت هرکدام (جمعا ۸ پایه)
S0..S4	خروجی	۱ بیت هرکدام (جمعا ۵ پایه)

زنجیره‌ی ripple-carry از ۴ نمونه‌ی full_adder: بیت صفر یک نیمه‌جمع‌کننده است (رقم نقلی ورودی‌اش مستقیماً به یک قطعه‌ی Constant با مقدار 0x0 وصل می‌شود)، و رقم نقلی خروجی هر طبقه به رقم نقلی ورودی طبقه‌ی بعد می‌رود. رقم نقلی خروجی طبقه‌ی آخر همان بیت پرارزش خروجی، S_4 ، است - چون $9 + 9 = 18$ در ۴ بیت جا نمی‌شود، خروجی باید ۵ بیت باشد.

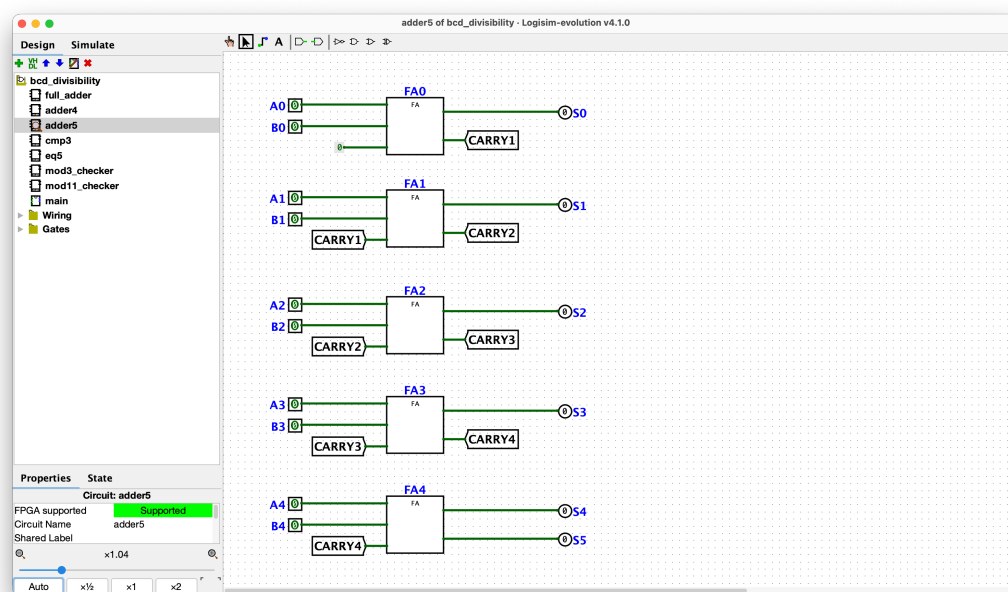
instance	bit	inputs	sum out	carry out
FA0	0 (LSB)	A_0, B_0, GND	S_0	C_1
FA1	1	A_1, B_1, C_1	S_1	C_2
FA2	2	A_2, B_2, C_2	S_2	C_3
FA3	3 (MSB)	A_3, B_3, C_3	S_3	S_4



جمع‌کننده ۵ بیتی (adder5)

پایه	جهت	پهنای
B0..B4 , A0..A4	ورودی	۱ بیت هرکدام (جمعا ۱۰ پایه)
S0..S5	خروجی	۱ بیت هرکدام (جمعا ۶ پایه)

دقیقا همان الگوی adder4، فقط با ۵ نمونه‌ی full_adder به جای ۴ (رقم نقلی ورودی طبقه‌ی صفر باز هم به GND). دو کاربرد در این آزمایش: در mod3_checker برای جمع نهایی دو مجموع ۵ بیتی S_{ab} و S_{cd} به یک عدد ۶ بیتی، و در mod11_checker برای جمع یک عدد ۵ بیتی با ثابت ۱۱ (BP11, AP11).

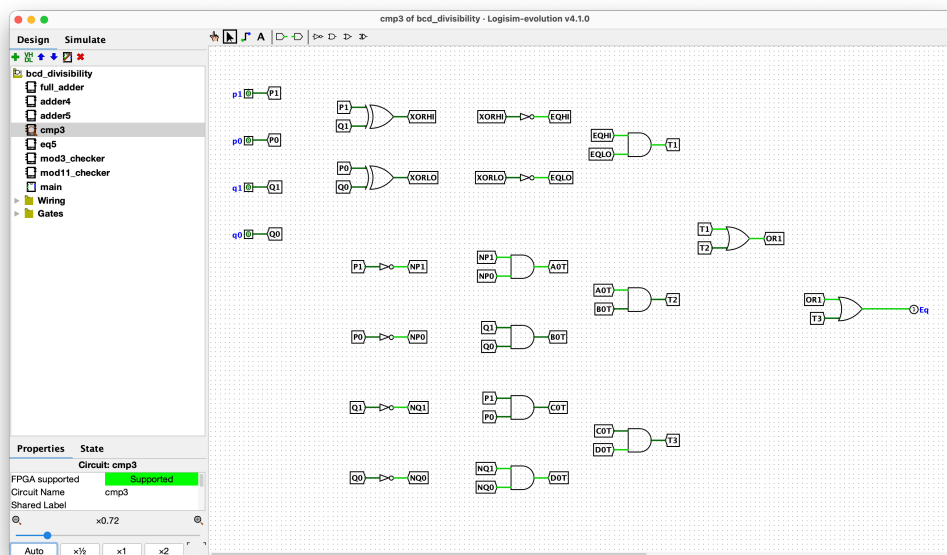


مقایسه‌گر $\text{mod } 3$ (cmp3)

پایه	جهت	پهنا
q_0, q_1, p_0, p_1	ورودی	۱ بیت هرکدام
Eq	خروجی	۱ بیت

شرط $P \bmod 3 = Q \bmod 3$ را برای $P, Q \in \{0, 1, 2, 3\}$ (بیت‌های $p_1 p_0$ و $q_1 q_0$) چک می‌کند - یعنی یا $P=Q$ ، یا $\{P, Q\} = \{0, 3\}$ (چون $3 \equiv 0 \pmod{3}$). نام‌گذاری سیگنال‌های زیر عیناً همان نام تونل‌هایی است که در مدار cmp3 داخل bcd_divisibility.circ استفاده شده:

- $XORLO = p_0 \oplus q_0$ ، $XORHI = p_1 \oplus q_1$ - دو XOR خام.
- $NQ_0 = \overline{q_0}$ ، $NQ_1 = \overline{q_1}$ ، $NP_0 = \overline{p_0}$ ، $NP_1 = \overline{p_1}$ - چهار NOT خام.
- $EQHI = \overline{XORHI}$ آیا $p_1 = q_1$ ؟ $EQLO = \overline{XORLO}$ آیا $p_0 = q_0$ ؟ (این دو با هم، XNOR روی گیت‌های پایه‌اند: NOT+XOR)
- $A_0T = NP_1 \wedge NP_0$ - شرط $P=0$ ، $B_0T = q_1 \wedge q_0$ - شرط $Q=3$.
- $C_0T = p_1 \wedge p_0$ - شرط $P=3$ ، $D_0T = NQ_1 \wedge NQ_0$ - شرط $Q=0$.
- $T_1 = EQHI \wedge EQLO$ - شرط $P=Q$.
- $T_2 = A_0T \wedge B_0T$ - شرط $P=0 \wedge Q=3$ ، $T_3 = C_0T \wedge D_0T$ - شرط $P=3 \wedge Q=0$.
- $Eq = OR_1 = T_1 \vee T_2 \vee T_3$ و در نهایت $OR_1 = T_1 \vee T_2 \vee T_3$.



p1	p0	q1	q0	Eq
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

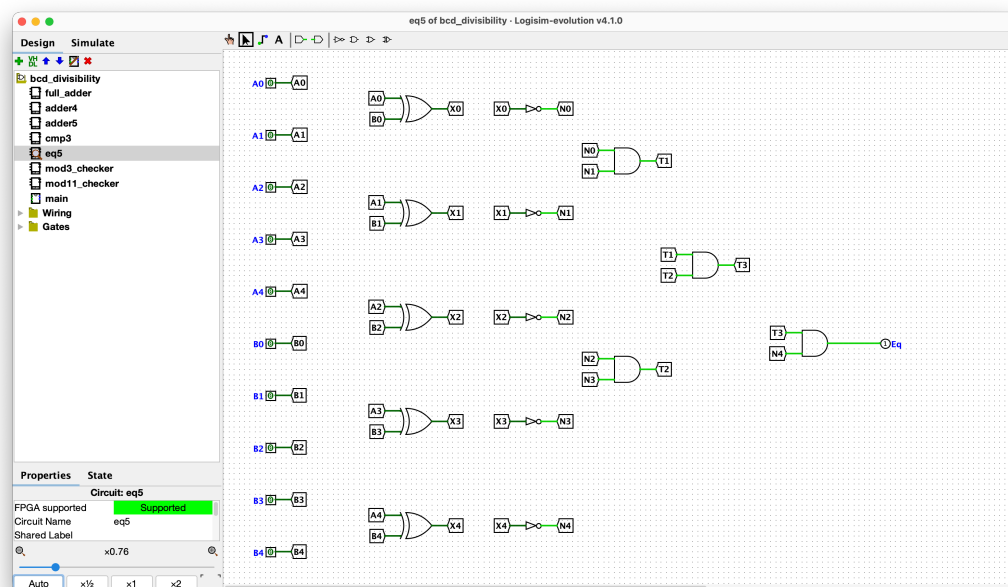
مقایسه گر تساوی ۵ بیتی (eq5)

پهنای	جهت	پایه
۱ بیت هرکدام (جمعا ۱۰ پایه)	ورودی	B0..B4, A0..A4
۱ بیت	خروجی	Eq

بررسی می‌کند آیا دو عدد ۵ بیتی A و B برابرند. نام‌گذاری زیر عینا همان نام تونل‌های مدار eq5 است: برای هر بیت i (۰ تا ۴) یک گیت XOR خام $X_i = A_i \oplus B_i$ ساخته می‌شود و بعد یک NOT روی آن، $N_i = \overline{X_i}$ (یعنی XNOR با گیت‌های پایه) — که ۱ است دقیقا وقتی $A_i = B_i$. سپس پنج نتیجه با یک درخت AND ترکیب می‌شوند:

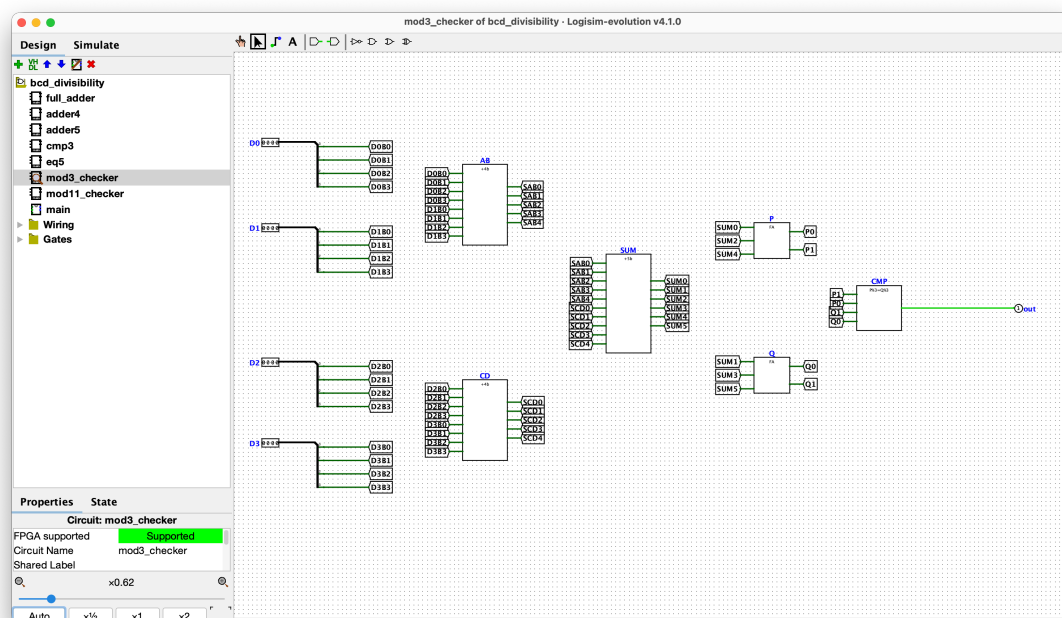
$$Eq = N_0 \wedge N_1 \wedge N_2 \wedge N_3 \wedge N_4.$$

درخت به صورت متوازن ساخته شده تا عمق مسیر ترکیبی کم بماند — دقیقا با همین نام‌های تونل در مدار: $T_1 = N_0 \wedge N_1$, $T_2 = N_2 \wedge N_3$, $T_3 = T_1 \wedge T_2$, و در نهایت $Eq = T_3 \wedge N_4$. در این آزمایش سه نمونه از eq5 در mod11_checker استفاده شده‌اند: $A=B$, $AP_{11}=B$ و $A=BP_{11}$.



معماری mod3_checker

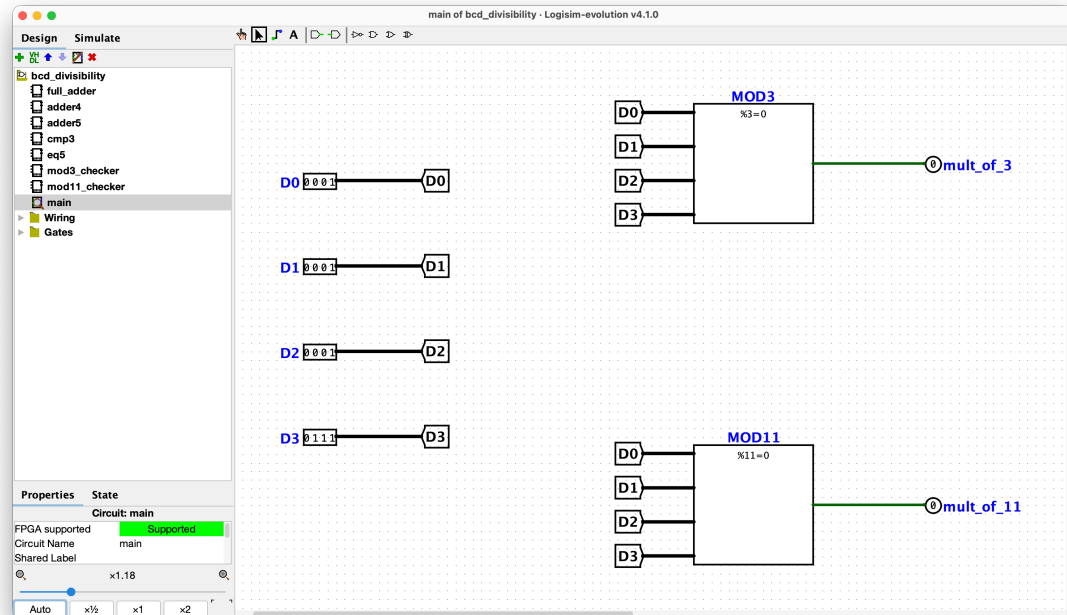
پایه	جهت	پهنا
D0, D1, D2, D3	ورودی	۴ بیت (رقم BCD)
out	خروجی	۱ بیت



$S = D_0 + D_1 + D_2 + D_3$ با adder5 ۵ بیتی به مجموع کل S تبدیل می‌شوند. سپس بیت‌های زوج S با یک full_adder جمع می‌شوند ($\rightarrow P$) و بیت‌های فرد با یکی دیگر ($\rightarrow Q$)، و شرط $P \bmod 3 = Q \bmod 3$ را چک می‌کند.

مدار main

برای مشاهده‌ی هم‌زمان هر دو خروجی، مدار main دو بلوک mod3_checker و mod11_checker را روی همان ۴ پایه‌ی ورودی مشترک $D_3 \dots D_0$ سوار می‌کند و دو خروجی mult_of_3 و mult_of_11 را جدا بیرون می‌دهد:



نتایج تست

هر ۸ مدار با بردار تست و از طریق خط فرمان Logisim در حالت headless (-test-vector) تست شدند. full_adder، adder4، adder5، cmp3، eq5 روی تمام فضای ورودی‌شان تست شدند، و mod11_checker/mod3_checker هر ۱۰۰۰۰ مقدار BCD ممکن (۰۰۰۰ تا ۹۹۹۹).

خروجی واقعی اجرای run_tests.sh:

```

circ : bcd_divisibility.circ
java : /opt/homebrew/opt/openjdk@21/bin/java
jar   : /Applications/Logisim-evolution.app/Contents/app/logisim-evolution-4.1.0-all.jar
vecs : 10

=== tests_adder4.vec → circuit 'adder4' ===
Loading test vector "tests_adder4..."vec
Running 256 ..vectors
Passed: 256, Failed: 0
PASS: tests_adder4.vec

=== tests_adder5.vec → circuit 'adder5' ===
Loading test vector "tests_adder5..."vec
Running 1024 ..vectors
Passed: 1024, Failed: 0
PASS: tests_adder5.vec

=== tests_cmp3.vec → circuit 'cmp3' ===
Loading test vector "tests_cmp3..."vec
Running 16 ..vectors
Passed: 16, Failed: 0
PASS: tests_cmp3.vec

=== tests_eq5.vec → circuit 'eq5' ===
Loading test vector "tests_eq5..."vec
Running 1024 ..vectors
Passed: 1024, Failed: 0
PASS: tests_eq5.vec

=== tests_full_adder.vec → circuit 'full_adder' ===
Loading test vector "tests_full_adder..."vec
Running 8 ..vectors
Passed: 8, Failed: 0
PASS: tests_full_adder.vec

=== tests_main.vec → circuit 'main' ===
Loading test vector "tests_main..."vec
Running 40 ..vectors
Passed: 40, Failed: 0
PASS: tests_main.vec

=== tests_mod11_checker_exhaustive.vec → circuit 'mod11_checker' ===
Loading test vector "tests_mod11_checker_exhaustive..."vec
Running 10000 ..vectors
Passed: 10000, Failed: 0
PASS: tests_mod11_checker_exhaustive.vec

=== tests_mod11_checker.vec → circuit 'mod11_checker' ===
Loading test vector "tests_mod11_checker..."vec
Running 35 ..vectors
Passed: 35, Failed: 0
PASS: tests_mod11_checker.vec

=== tests_mod3_checker_exhaustive.vec → circuit 'mod3_checker' ===
Loading test vector "tests_mod3_checker_exhaustive..."vec
Running 10000 ..vectors
Passed: 10000, Failed: 0
PASS: tests_mod3_checker_exhaustive.vec

=== tests_mod3_checker.vec → circuit 'mod3_checker' ===
Loading test vector "tests_mod3_checker..."vec
Running 35 ..vectors
Passed: 35, Failed: 0
PASS: tests_mod3_checker.vec

summary: 10/10 passed, 0 failed

```

چون هر دو مدار اصلی روی تمام فضای ورودی معتبر تست شده‌اند، درستی مدار برای هر عدد BCD چهاررقمی تضمین شده است.